

Relational Databases

Chapter 3 | PostgreSQL Complete Guide

Difficulty: Beginner-Intermediate | Estimated Reading Time: 30 minutes

Table of Contents

- [Learning Objectives](#)
- [Theory: The Relational Model](#)
- [Codd's 12 Rules](#)
- [Keys and Constraints](#)
- [Relationships Between Tables](#)
- [Normalization Overview](#)
- [ASCII Visual Diagrams](#)
- [Practical Examples](#)
- [PostgreSQL Commands](#)
- [SQL Examples](#)
- [Common Mistakes](#)
- [Best Practices](#)
- [Performance Considerations](#)
- [Interview Questions](#)
- [Interview Answers](#)
- [Hands-on Exercises](#)
- [Solutions](#)
- [Advanced Notes](#)
- [Cross-References](#)

Learning Objectives

By the end of this chapter, you will be able to:

- Explain the relational model as defined by E.F. Codd
- Identify and implement different types of keys (primary, foreign, composite, surrogate)
- Model the three types of relationships: one-to-one, one-to-many, many-to-many
- Apply the first three normal forms to eliminate data anomalies
- Design a normalized relational schema for a real-world problem

Theory: The Relational Model

The **relational model** was proposed by British computer scientist **Edgar F. Codd** in his landmark 1970 paper *"A Relational Model of Data for Large Shared Data Banks."* It remains the dominant data model for structured data storage over 50 years later.

Core Concepts

Term	Mathematical Equivalent	Database Equivalent
Relation	Set	Table
Tuple	Element of a set	Row / Record

Attribute	Domain	Column / Field
Domain	Data type	Column's allowed values
Degree	Number of attributes	Number of columns
Cardinality	Number of tuples	Number of rows

Properties of a Relation (Table)

- 1. Each row is unique (no duplicate tuples)
- 2. Each cell contains exactly one atomic value (First Normal Form)
- 3. Column order does not matter
- 4. Row order does not matter
- 5. Each column has a unique name
- 6. All values in a column are from the same domain (data type)

Relational Algebra

The formal mathematical foundation of SQL consists of these operations:

Operation	SQL Equivalent	Description
Selection (σ)	WHERE	Filter rows
Projection (π)	SELECT columns	Filter columns
Union ($\cup$)	UNION	Combine tuples from two relations
Difference ($-$)	EXCEPT	Tuples in first but not second
Intersection ($\cap$)	INTERSECT	Tuples in both
Cartesian Product ($\times$)	CROSS JOIN	All combinations of rows
Join ($\bowtie$)	JOIN	Combine based on condition
Division ($\div$)	No direct equivalent	Complex division queries

Codd's 12 Rules

In 1985, Codd published 12 rules that a database must satisfy to be considered "relational." These rules remain the gold standard:

Rule	Name	Description
0	Foundation Rule	Must manage data using relational capabilities only
1	Information Rule	All data represented as values in tables
2	Guaranteed Access	Every data item accessible via table + PK + column
3	Systematic NULL Treatment	NULLs represent missing/inapplicable data uniformly
4	Dynamic Online Catalog	Schema stored in the database itself (system tables)

5	Comprehensive Data Sublanguage	Must support at least one relational language (SQL)
6	View Updating Rule	All theoretically updatable views must be updatable
7	High-Level Insert/Update/Delete	Must support set-level operations, not just row-level
8	Physical Data Independence	Application immune to changes in storage
9	Logical Data Independence	Application immune to schema changes (views protect)
10	Integrity Independence	Integrity constraints in DB, not application code
11	Distribution Independence	Works the same regardless of physical distribution
12	Non-Subversion Rule	Low-level access cannot bypass relational rules

Keys and Constraints

Types of Keys

Super Key: Any combination of columns that uniquely identifies a row.

Candidate Key: A minimal super key — no column can be removed and still maintain uniqueness.

Primary Key (PK): The chosen candidate key. Must be NOT NULL and UNIQUE. Every table should have one.

Foreign Key (FK): A column that references the primary key of another table, establishing a relationship.

Composite Key: A primary key made of two or more columns.

Surrogate Key: An artificial key (usually auto-incremented integer or UUID) with no business meaning.

Natural Key: A key derived from actual business data (e.g., SSN, email, ISBN).

Surrogate vs. Natural Keys

Aspect	Surrogate Key	Natural Key
Stability	Never changes	May change (email, SSN)
Meaning	No business meaning	Has business context
Join Performance	Faster (integer)	Slower (string comparison)
Debugging	Harder to interpret	Easier to understand
Best Practice	Preferred for PK	Use for UNIQUE constraints

Constraints in PostgreSQL

```
CREATE TABLE employees (
  employee_id SERIAL PRIMARY KEY,           -- Primary Key
  email VARCHAR(100) UNIQUE NOT NULL,      -- Unique + Not Null
  department_id INT REFERENCES departments(id), -- Foreign Key
  salary DECIMAL(10,2) CHECK (salary > 0), -- Check constraint
```

```
status      VARCHAR(20) DEFAULT 'active',      -- Default value
hire_date   DATE      NOT NULL                  -- Not Null
);
```

Relationships Between Tables

One-to-Many (1:N) — Most Common

One record in table A relates to many records in table B.

Example: One customer can have many orders.

```
customers (1) -----> (N) orders
```

One-to-One (1:1)

One record in table A relates to exactly one record in table B.

Example: One employee has one employee_profile (for large optional data).

```
employees (1) -----> (1) employee_profiles
```

Usually implemented by putting the FK in the "dependent" table.

Many-to-Many (M:N)

Many records in table A relate to many records in table B.

Example: A student can enroll in many courses; a course has many students.

```
students (M) -----> (N) courses
```

Implemented via a **junction table** (also called bridge table or associative table).

```
students <--- student_courses ---> courses
```

Normalization Overview

Normalization is the process of organizing data to reduce redundancy and improve integrity.

First Normal Form (1NF)

- Every column contains atomic (indivisible) values
- No repeating groups or arrays
- Each row is unique

Violation: phone_numbers = "555-1234, 555-5678" — multiple values in one cell. **Fix:** Create a separate phone_numbers table.

Second Normal Form (2NF)

- Must be in 1NF

- Every non-key column is fully functionally dependent on the entire primary key (no partial dependencies)
- Only relevant when using composite primary keys

Third Normal Form (3NF)

- Must be in 2NF
- No transitive dependencies: non-key columns should not depend on other non-key columns

Example of transitive dependency:

```
orders(order_id, customer_id, customer_name, customer_city)
      PK           depends on customer_id, not order_id
```

Fix: Move customer_name and customer_city to a customers table.

Boyce-Codd Normal Form (BCNF)

A stricter version of 3NF. For every functional dependency $X \rightarrow Y$, X must be a superkey.

ASCII Visual Diagrams

Diagram 1: Relational Table Anatomy

TABLE: orders

=====

order_id	customer_id	order_date	total	status
(PK)	(FK)			

1001	42	2024-01-15	129.99	shipped
1002	17	2024-01-16	49.50	pending
1003	42	2024-01-17	299.00	shipped

<-- Tuple (Row)

^

^

|

|

Primary Key

Foreign Key (references customers.id)

Degree (columns) = 5

Cardinality (rows) = 3

Diagram 2: Entity Relationship (ER) Diagram

CUSTOMERS

=====

[customer_id] PK

first_name

last_name

email UNIQUE

phone

created_at

|

ORDERS

=====

[order_id] PK

customer_id FK

order_date

total

status

|

ORDER_ITEMS (Junction)

PRODUCTS

=====

[product_id] PK

name

price

stock_qty

category_id FK

```

| 1:N | 1:N | =====
|      |      | [order_id] FK -> orders
|      |      | [product_id] FK -> products
+-----+-----> quantity
                    unit_price
                    (PK = order_id + product_id)

```

CATEGORIES

=====

[category_id] PK

name

description

Diagram 3: Normalization Steps

BEFORE NORMALIZATION (Problematic)

=====

order_id	customer	cust_city	product	qty	price
1	Alice	NYC	Laptop	1	999
1	Alice	NYC	Mouse	2	29
2	Bob	LA	Laptop	1	999

PROBLEMS:

- "Alice" and "NYC" repeated (redundancy)
- If Alice moves, must update multiple rows (update anomaly)
- Can't store a customer without an order (insertion anomaly)
- Deleting last order for Bob deletes Bob's record (deletion anomaly)

AFTER 3NF (Clean)

=====

customers		order_items			products	
cust_id	name	order_id	prod_id	prod_id	name	price
1	Alice	1	1	1	Laptop	999
2	Bob	2	2	2	Mouse	29

Practical Examples

Example 1: University Database

ENTITIES: Students, Courses, Professors, Departments

RELATIONSHIPS:

- Department has many Professors (1:N)
- Department offers many Courses (1:N)

- Professor teaches many Courses (1:N)
- Student enrolls in many Courses; Course has many Students (M:N)
 - > Junction: enrollments(student_id, course_id, grade, enrolled_date)

Example 2: Hospital Database

ENTITIES: Patients, Doctors, Appointments, Diagnoses, Prescriptions

RELATIONSHIPS:

- Doctor has many Appointments (1:N)
- Patient has many Appointments (1:N)
- Appointment has one Doctor and one Patient (from FK perspective)
- Patient can have many Diagnoses (1:N)
- Diagnosis can have many Prescriptions (1:N)

PostgreSQL Commands

```
-- View all constraints on a table
SELECT conname, contype, conrelid::regclass, pg_get_constraintdef(oid)
FROM pg_constraint
WHERE conrelid = 'orders'::regclass;

-- View foreign key relationships
SELECT
    tc.table_name,
    kcu.column_name,
    ccu.table_name AS foreign_table,
    ccu.column_name AS foreign_column
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu ON tc.constraint_name =
kcu.constraint_name
JOIN information_schema.constraint_column_usage AS ccu ON ccu.constraint_name =
tc.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY';

-- Check table sizes and row counts
SELECT
    relname AS table_name,
    n_live_tup AS row_count,
    pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM pg_stat_user_tables
ORDER BY n_live_tup DESC;
```

SQL Examples

Example 1: Creating a Normalized Schema

```

-- Departments table
CREATE TABLE departments (
    dept_id SERIAL PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    budget DECIMAL(15, 2)
);

-- Employees table
CREATE TABLE employees (
    emp_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    dept_id INT REFERENCES departments(dept_id) ON DELETE SET NULL,
    salary DECIMAL(10, 2) CHECK (salary > 0),
    hire_date DATE NOT NULL DEFAULT CURRENT_DATE
);

```

Example 2: Many-to-Many with Junction Table

```

CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE
);

CREATE TABLE courses (
    course_id SERIAL PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    credits INT CHECK (credits BETWEEN 1 AND 6)
);

-- Junction table
CREATE TABLE enrollments (
    student_id INT REFERENCES students(student_id) ON DELETE CASCADE,
    course_id INT REFERENCES courses(course_id) ON DELETE CASCADE,
    enrolled_at TIMESTAMP DEFAULT NOW(),
    grade CHAR(2),
    PRIMARY KEY (student_id, course_id)
);

```

Example 3: Self-Referential Table (Hierarchy)

```

CREATE TABLE categories (
    category_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    parent_id INT REFERENCES categories(category_id)
);

```

```
INSERT INTO categories (name, parent_id) VALUES
('Electronics', NULL),
('Computers', 1),
('Laptops', 2),
('Gaming Laptops', 3);
```

Example 4: Enforcing Business Rules with CHECK

```
CREATE TABLE products (
  product_id SERIAL PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  price DECIMAL(10, 2) NOT NULL CHECK (price >= 0),
  discount_pct DECIMAL(5, 2) CHECK (discount_pct BETWEEN 0 AND 100),
  launch_date DATE,
  end_date DATE,
  CONSTRAINT valid_dates CHECK (end_date IS NULL OR end_date > launch_date)
);
```

Example 5: INNER JOIN — The Core of Relational Power

```
-- Get all orders with customer names
SELECT
  c.first_name || ' ' || c.last_name AS customer_name,
  o.order_id,
  o.order_date,
  o.total
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
ORDER BY o.order_date DESC;
```

Example 6: LEFT JOIN — Include Customers Without Orders

```
SELECT
  c.first_name,
  c.last_name,
  COUNT(o.order_id) AS order_count,
  COALESCE(SUM(o.total), 0) AS total_spent
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_spent DESC;
```

Example 7: Three-Table JOIN

```
SELECT
  s.name AS student,
  c.title AS course,
```

```
    e.grade
FROM enrollments e
JOIN students s ON e.student_id = s.student_id
JOIN courses c ON e.course_id = c.course_id
WHERE e.grade IS NOT NULL;
```

Example 8: Referential Integrity in Action

```
-- ON DELETE CASCADE: deleting a student removes their enrollments
-- ON DELETE SET NULL: deleting a dept sets emp.dept_id to NULL
-- ON DELETE RESTRICT (default): prevents deletion if referenced rows exist

-- Try to delete a department with employees (will fail with RESTRICT)
-- DELETE FROM departments WHERE dept_id = 1; -- ERROR!

-- With CASCADE, dependent rows are deleted automatically
ALTER TABLE orders
ADD CONSTRAINT fk_customer
FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
ON DELETE CASCADE;
```

Example 9: Adding Constraints After Table Creation

```
-- Add a unique constraint
ALTER TABLE employees ADD CONSTRAINT uq_employee_email UNIQUE (email);

-- Add a check constraint
ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (salary > 0);

-- Add a foreign key
ALTER TABLE employees ADD CONSTRAINT fk_dept
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id);
```

Example 10: Finding Orphaned Records

```
-- Find orders with no matching customer (data integrity check)
SELECT o.order_id, o.customer_id
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
```

Example 11: Checking Normalization Violations

```
-- Find customers appearing in orders table (transitive dependency check)
-- This query tests if customer data is properly normalized
SELECT DISTINCT customer_id
```

```
FROM orders
WHERE customer_id NOT IN (SELECT customer_id FROM customers);
```

Example 12: Entity Count Per Relationship

```
-- How many orders per customer?
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 5
ORDER BY order_count DESC;
```

Common Mistakes

Mistake 1: Missing Foreign Key Constraints

Relying on application code to maintain referential integrity instead of database-level FK constraints. This leads to orphaned records.

Mistake 2: Using VARCHAR Primary Keys

```
-- BAD: String PK is slow for joins and may change
CREATE TABLE users (username VARCHAR(50) PRIMARY KEY, ...);

-- GOOD: Integer surrogate key, username gets UNIQUE constraint
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    ...
);
```

Mistake 3: Storing Comma-Separated Values

```
-- BAD: Violates 1NF
CREATE TABLE posts (tags VARCHAR(500)); -- "tech,sql,database"

-- GOOD: Separate table or PostgreSQL ARRAY/JSONB
CREATE TABLE post_tags (post_id INT, tag VARCHAR(50));
```

Mistake 4: Not Naming Constraints

```
-- BAD: Auto-generated constraint name is hard to manage
ALTER TABLE orders ADD FOREIGN KEY (customer_id) REFERENCES customers(id);

-- GOOD: Named constraint for clear error messages and management
```

```
ALTER TABLE orders ADD CONSTRAINT fk_orders_customer
FOREIGN KEY (customer_id) REFERENCES customers(id);
```

Mistake 5: Over-Normalization

Normalization is beneficial, but splitting data into too many tables can hurt read performance. Sometimes controlled denormalization (with redundancy managed via triggers or materialized views) is acceptable for high-read workloads.

Mistake 6: Forgetting to Index Foreign Keys

Foreign keys are not automatically indexed in PostgreSQL. An un-indexed FK causes full table scans during joins.

```
-- Always add an index on foreign key columns
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_enrollments_student_id ON enrollments(student_id);
CREATE INDEX idx_enrollments_course_id ON enrollments(course_id);
```

Best Practices

1. **Every table needs a primary key** — Prefer surrogate integer/UUID PKs.
2. **Name all constraints** — `CONSTRAINT fk_orders_customer` is better than auto-generated names.
3. **Index all foreign key columns** — PostgreSQL does not auto-index FKs.
4. **Use ON DELETE actions** — Explicitly define CASCADE, SET NULL, or RESTRICT based on business rules.
5. **Normalize to 3NF by default** — Denormalize only when there is a measured performance need.
6. **Document your ERD** — Maintain an up-to-date Entity-Relationship Diagram.
7. **Use appropriate data types** — Don't store numbers as VARCHAR; don't store dates as TEXT.

Performance Considerations

- **JOIN performance** depends on indexes on join columns. Always index FK columns.
- **Selectivity matters:** High-cardinality columns (many unique values) benefit most from B-tree indexes.
- **Covering indexes:** An index that includes all queried columns avoids heap access (index-only scan).
- **Denormalization for reads:** Materialized views can pre-compute expensive JOINS.
- **Partitioning large tables** improves query performance on date-range queries.

```
-- Check if FK indexes exist
SELECT
    t.relname AS table_name,
    a.attname AS column_name,
    i.relname AS index_name
FROM pg_class t
JOIN pg_attribute a ON a.attrelid = t.oid
LEFT JOIN pg_index x ON x.indrelid = t.oid AND a.attnum = ANY(x.indkey)
LEFT JOIN pg_class i ON i.oid = x.indexrelid
WHERE t.relkind = 'r'
```

```
AND t.relname = 'orders'
ORDER BY column_name;
```

Interview Questions

1. What is the relational model and who invented it?
 2. What is the difference between a primary key and a foreign key?
 3. What is a candidate key?
 4. What is the difference between a surrogate key and a natural key?
 5. Explain the three types of relationships in a relational database.
 6. What are the first three normal forms?
 7. What is referential integrity?
 8. Why should foreign key columns be indexed in PostgreSQL?
 9. What is a self-referential (recursive) relationship?
 10. What are the ON DELETE actions for foreign keys?
-

Interview Answers

Q1: E.F. Codd invented the relational model in 1970. It organizes data into tables (relations) of rows (tuples) and columns (attributes), with mathematical foundations in set theory and relational algebra.

Q3: Candidate Key

A candidate key is a minimal set of attributes that uniquely identifies each tuple. "Minimal" means no attribute can be removed while still maintaining uniqueness. There can be multiple candidate keys; one is chosen as the primary key, the rest become alternate keys (implemented as UNIQUE constraints).

Q6: First Three Normal Forms

- **1NF:** Atomic values, no repeating groups, unique rows
- **2NF:** 1NF + no partial dependencies on composite PK (every non-key column depends on the whole PK)
- **3NF:** 2NF + no transitive dependencies (non-key columns only depend on the PK, not on other non-key columns)

Q10: ON DELETE Actions

- **CASCADE** : Delete dependent rows automatically
 - **SET NULL** : Set FK column to NULL when referenced row is deleted
 - **SET DEFAULT** : Set FK column to its default value
 - **RESTRICT** : Prevent deletion if dependent rows exist (checked at end of statement)
 - **NO ACTION** (default): Like RESTRICT but checked at end of transaction
-

Hands-on Exercises

Exercise 1: Design a Library Schema

Design tables for: Books, Authors, Members, Loans. Include proper PKs, FKs, and constraints. A book can have multiple authors; a member can borrow multiple books.

Exercise 2: Normalization

Given the following table, normalize it to 3NF: `order_data(order_id, product_id, product_name, product_price, customer_id, customer_name, customer_email, qty, line_total)`

Exercise 3: Implement Relationships

Create the library schema from Exercise 1 in PostgreSQL, insert sample data, and write queries joining all four tables.

Exercise 4: Referential Integrity Test

Create a parent-child table pair. Try to insert a child record with a non-existent parent. Observe the error. Then test ON DELETE CASCADE.

Exercise 5: Find Schema Issues

Given a table without proper normalization, identify all violations and propose fixes.

Solutions

Solution 1 (Library Schema Design)

```
CREATE TABLE authors (  
    author_id SERIAL PRIMARY KEY,  
    name      VARCHAR(150) NOT NULL,  
    bio       TEXT  
);  
  
CREATE TABLE books (  
    book_id   SERIAL PRIMARY KEY,  
    isbn      VARCHAR(20) UNIQUE NOT NULL,  
    title     VARCHAR(300) NOT NULL,  
    published DATE,  
    copies_qty INT NOT NULL DEFAULT 1 CHECK (copies_qty >= 0)  
);  
  
CREATE TABLE book_authors (  
    book_id   INT REFERENCES books(book_id) ON DELETE CASCADE,  
    author_id INT REFERENCES authors(author_id) ON DELETE CASCADE,  
    PRIMARY KEY (book_id, author_id)  
);  
  
CREATE TABLE members (  
    member_id SERIAL PRIMARY KEY,  
    name      VARCHAR(150) NOT NULL,  
    email     VARCHAR(100) UNIQUE NOT NULL,  
    joined_date DATE DEFAULT CURRENT_DATE  
);  
  
CREATE TABLE loans (  
    loan_id   SERIAL PRIMARY KEY,  
    book_id   INT NOT NULL REFERENCES books(book_id),  
    member_id INT NOT NULL REFERENCES members(member_id),
```

```
    loaned_at    TIMESTAMP DEFAULT NOW(),
    due_date     DATE NOT NULL,
    returned_at  TIMESTAMP
);
```

Solution 2 (Normalize to 3NF)

```
-- 3NF normalization of order_data
CREATE TABLE products (
    product_id    INT PRIMARY KEY,
    product_name  VARCHAR(200),
    product_price DECIMAL(10,2)
);
CREATE TABLE customers (
    customer_id   INT PRIMARY KEY,
    customer_name VARCHAR(100),
    customer_email VARCHAR(100)
);
CREATE TABLE orders (
    order_id      INT PRIMARY KEY,
    customer_id   INT REFERENCES customers(customer_id)
);
CREATE TABLE order_lines (
    order_id      INT REFERENCES orders(order_id),
    product_id    INT REFERENCES products(product_id),
    qty           INT,
    line_total    DECIMAL(10,2),
    PRIMARY KEY (order_id, product_id)
);
```

Advanced Notes

Functional Dependencies

A **functional dependency** $X \rightarrow Y$ means: knowing the value of X uniquely determines the value of Y. This is the mathematical basis of normalization.

Example: `student_id` $\rightarrow$ `student_name` (knowing the ID tells you the name)

Denormalization Patterns

Common read-optimization patterns:

- **Materialized views:** Pre-computed joins stored as tables
- **Derived columns:** Storing computed values (`total_price = qty * unit_price` stored as a column)
- **Report tables:** Denormalized summary tables refreshed periodically

PostgreSQL-Specific Features

```
-- Deferrable constraints (checked at commit, not statement level)
ALTER TABLE orders ADD CONSTRAINT fk_customer
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    DEFERRABLE INITIALLY DEFERRED;

-- Exclusion constraints (generalization of UNIQUE)
CREATE TABLE reservations (
    room_id INT,
    during TSRANGE,
    EXCLUDE USING GIST (room_id WITH =, during WITH &&)
);
-- Prevents overlapping reservations for the same room
```

Cross-References

- **Previous:** [02 - Types of Databases](#)
- **Next:** [04 - OLTP vs OLAP](#)
- **Related:** [06 - ACID Properties](#) — Transactions that protect relational integrity
- **See Also:** [02 SQL Basics/01 ddl commands.md](#) — CREATE, ALTER, DROP commands
- **See Also:** [04 Advanced SQL/03 joins.md](#) — Advanced JOIN techniques
- **See Also:** [05 Schema Design/01 normalization.md](#) — Full normalization guide